

1. Objective

To expose a trained deep learning model as a REST API using the Flask framework, so that predictions can be requested over HTTP from any client - a web page, a mobile app, curl, Postman, or another program - without needing direct access to the model file or the Python code that trains it.

2. Reference Model

The API serves the CNN digit-classification model trained from scratch in Task 1 (`cnn_digits_model.pkl`), which achieved 97.04% accuracy on its held-out test set. The Flask API loads this saved model once at startup and reuses it for every incoming request, rather than reloading it each time.

3. Approach

1. **Load the trained model** - the saved convolutional and dense layer weights from Task 1 are loaded once when the Flask application starts.
2. **Create prediction endpoints** - a single-image endpoint (`POST /predict`) and a batch endpoint (`POST /predict/batch`), plus an information endpoint (`GET /`) and a health-check endpoint (`GET /health`).
3. **Accept input data and return predictions** - each endpoint accepts a JSON request body containing image pixel data and returns the predicted digit, a confidence score, and the full probability distribution over all ten digits, as JSON.
4. **Implement JSON request-response handling** - both the request parsing and the response formatting are done entirely in JSON, with input validation performed before any prediction is attempted.
5. **Add exception and error handling** - invalid JSON, missing fields, wrong-size inputs, unknown endpoints, and wrong HTTP methods are all caught and turned into structured JSON error responses with appropriate HTTP status codes (400, 404, 405), and any unexpected internal error is caught and logged rather than crashing the server or leaking a stack trace to the client.

4. Source Code - Flask Application (app.py)

The complete Flask application (also provided as the standalone file `app.py` , runnable directly with `python app.py` , and demonstrated interactively in the accompanying notebook `Task4_Flask_API_Demo.ipynb`):

```
"""
Task 4: Developing a Flask API for Deep Learning Models
=====
Exposes the CNN digit-classification model trained in Task 1 (a from-scratch
NumPy CNN, saved as cnn_digits_model.pkl) as a REST API using Flask.

Endpoints:
GET /           - API information
GET /health     - health check
POST /predict   - predict the digit for a single 8x8 grayscale image
POST /predict/batch - predict digits for multiple images in one request

Run with: python app.py
Then send requests to http://127.0.0.1:5000/
"""

import os
import pickle
import traceback
import numpy as np
from flask import Flask, request, jsonify

MODEL_PATH = os.path.join(os.path.dirname(__file__), "cnn_digits_model.pkl")

app = Flask(__name__)

# -----
# Model definition (must match the architecture used to train and save
# cnn_digits_model.pkl in Task 1) and inference-only forward pass.
# -----

def im2col(x, kh, kw, stride=1, pad=0):
    N, H, W, C = x.shape
    if pad > 0:
        x = np.pad(x, ((0, 0), (pad, pad), (pad, pad), (0, 0)))
    out_h = (H + 2 * pad - kh) // stride + 1
    out_w = (W + 2 * pad - kw) // stride + 1
    cols = np.zeros((N, out_h, out_w, kh, kw, C), dtype=x.dtype)
    for i in range(kh):
        i_max = i + stride * out_h
        for j in range(kw):
            j_max = j + stride * out_w
            cols[:, :, :, i, j, :] = x[:, i:i_max:stride, j:j_max:stride, :]
```

```

return cols.reshape(N, out_h, out_w, kh * kw * C), out_h, out_w

def conv_forward(x, W, b, stride=1, pad=1):
    N, H, Wd, C = x.shape
    k = W.shape[0]
    out_ch = W.shape[-1]
    cols, out_h, out_w = im2col(x, k, k, stride, pad)
    W_col = W.reshape(-1, out_ch)
    out = cols.reshape(N * out_h * out_w, -1) @ W_col + b
    return out.reshape(N, out_h, out_w, out_ch)

def relu(x):
    return np.maximum(0, x)

def maxpool_forward(x, size=2, stride=2):
    N, H, W, C = x.shape
    out_h, out_w = H // stride, W // stride
    x = x[:, :out_h * stride, :out_w * stride, :]
    out = np.zeros((N, out_h, out_w, C), dtype=x.dtype)
    for i in range(out_h):
        for j in range(out_w):
            window = x[:, i*stride:i*stride+size, j*stride:j*stride+size, :]
            out[:, i, j, :] = window.max(axis=(1, 2))
    return out

def softmax(logits):
    e = np.exp(logits - logits.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

class DigitCNN:
    """Loads the Task 1 CNN weights and runs inference only (no training)."""

    def __init__(self, weights_path):
        with open(weights_path, "rb") as f:
            self.w = pickle.load(f)

    def predict_proba(self, x):
        """x: numpy array of shape (N, 8, 8, 1), pixel values already in [0, 1]."""
        w = self.w
        x = conv_forward(x, w["conv1_W"], w["conv1_b"], stride=1, pad=1)
        x = relu(x)
        x = maxpool_forward(x, 2, 2)
        x = conv_forward(x, w["conv2_W"], w["conv2_b"], stride=1, pad=1)
        x = relu(x)
        x = maxpool_forward(x, 2, 2)
        x = x.reshape(x.shape[0], -1)
        x = x @ w["fc1_W"] + w["fc1_b"]
        x = relu(x)
        logits = x @ w["fc2_W"] + w["fc2_b"]
        return softmax(logits)

# Load the model once at startup, not on every request.
model = DigitCNN(MODEL_PATH)

# -----
# Helpers
# -----

def parse_image(payload):
    """
    Validates and converts a single image payload into a (1, 8, 8, 1)
    float32 array normalized to [0, 1]. Raises ValueError with a clear
    message on any problem, which the route handlers turn into a 400.
    """
    if payload is None:
        raise ValueError("Missing 'image' field in request body.")

    arr = np.array(payload, dtype=np.float64)

    if arr.size != 64:
        raise ValueError(
            f"Expected an 8x8 (64-value) grayscale image, got {arr.size} values."
        )

    arr = arr.reshape(8, 8)

    if np.isnan(arr).any():
        raise ValueError("Image contains non-numeric or missing values.")

    # Accept either raw 0-16 pixel scale (like the original digits dataset)
    # or already-normalized 0-1 scale, and normalize consistently to [0, 1].
    if arr.max() > 1.0:
        if arr.max() > 16.0 or arr.min() < 0.0:
            raise ValueError("Pixel values must be within [0, 16] (or already normalized to [0, 1]).")
        arr = arr / 16.0
    elif arr.min() < 0.0:

```

```

        raise ValueError("Pixel values must not be negative.")

    return arr.reshape(1, 8, 8, 1).astype(np.float32)

def format_prediction(probs_row):
    probs_row = probs_row.tolist()
    pred_digit = int(np.argmax(probs_row))
    return {
        "predicted_digit": pred_digit,
        "confidence": round(probs_row[pred_digit], 4),
        "probabilities": {str(i): round(p, 4) for i, p in enumerate(probs_row)},
    }

# -----
# Routes
# -----

@app.route("/", methods=["GET"])
def index():
    return jsonify({
        "name": "Digit Classification API",
        "description": "REST API serving the Task 1 from-scratch CNN digit classifier.",
        "endpoints": {
            "GET /health": "Health check.",
            "POST /predict": "Predict the digit for one 8x8 grayscale image. "
                             "Body: {\\"image\\": [64 numbers, 0-16 or 0-1, row-major 8x8]}",
            "POST /predict/batch": "Predict digits for multiple images. "
                                  "Body: {\\"images\\": [[64 numbers], [64 numbers], ...]}",
        },
    })

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok", "model_loaded": model is not None})

@app.route("/predict", methods=["POST"])
def predict():
    try:
        data = request.get_json(silent=True)
        if data is None:
            return jsonify({"error": "Request body must be valid JSON."}), 400

        image = parse_image(data.get("image"))
        probs = model.predict_proba(image)[0]
        return jsonify(format_prediction(probs)), 200

    except ValueError as e:
        return jsonify({"error": str(e)}), 400
    except Exception as e:
        app.logger.error("Unexpected error in /predict: %s\\n%s", e, traceback.format_exc())
        return jsonify({"error": "Internal server error while generating prediction."}), 500

@app.route("/predict/batch", methods=["POST"])
def predict_batch():
    try:
        data = request.get_json(silent=True)
        if data is None:
            return jsonify({"error": "Request body must be valid JSON."}), 400

        images_payload = data.get("images")
        if not isinstance(images_payload, list) or len(images_payload) == 0:
            return jsonify({"error": "'images' must be a non-empty list of 64-value images."}), 400
        if len(images_payload) > 100:
            return jsonify({"error": "Batch size limited to 100 images per request."}), 400

        results = []
        for idx, img_payload in enumerate(images_payload):
            try:
                image = parse_image(img_payload)
                probs = model.predict_proba(image)[0]
                results.append(format_prediction(probs))
            except ValueError as e:
                results.append({"error": f"image[{idx}]: {e}"})

        return jsonify({"count": len(results), "results": results}), 200

    except Exception as e:
        app.logger.error("Unexpected error in /predict/batch: %s\\n%s", e, traceback.format_exc())
        return jsonify({"error": "Internal server error while generating predictions."}), 500

# -----
# Error handlers for common HTTP errors
# -----

@app.errorhandler(404)
def not_found(e):
    return jsonify({"error": "Endpoint not found. See GET / for a list of available endpoints."}), 404

```

```

@app.errorhandler(405)
def method_not_allowed(e):
    return jsonify({"error": "Method not allowed for this endpoint."}), 405

@app.errorhandler(500)
def server_error(e):
    return jsonify({"error": "Internal server error."}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)

```

5. API Documentation Summary

Endpoint	Method	Purpose
/	GET	API information and list of endpoints
/health	GET	Health check - confirms the model loaded successfully
/predict	POST	Predict the digit for one 8x8 grayscale image
/predict/batch	POST	Predict digits for up to 100 images in one request

Full request/response formats, status codes, and usage examples (curl and Python `requests`) are provided in the accompanying `README.md`.

6. Screenshots / Sample Outputs

6.1 Sample Test Images

Three genuine images from the scikit-learn Digits dataset (true labels 3, 7, and 8) were sent to the API to verify real predictions end-to-end.

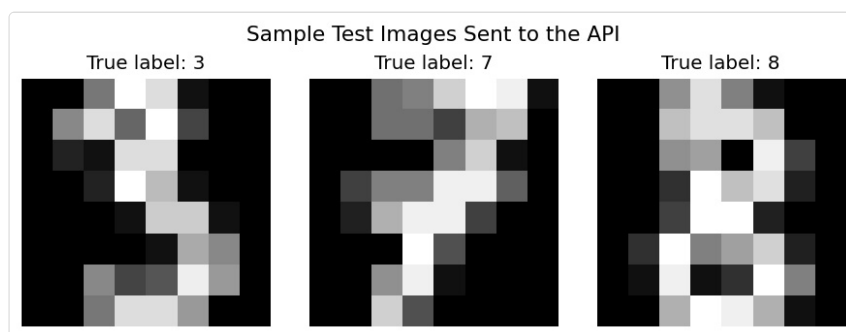


Figure 1: The three real digit images used to test the `/predict` and `/predict/batch` endpoints.

6.2 Live API Test Output (Console Log)

The following is the actual console output captured while the Flask server was running and being called with curl - covering the information endpoint, health check, a successful prediction, a successful batch prediction on real digit images, and every implemented error case:

```

=== GET / ===
{
  "description": "REST API serving the Task 1 from-scratch CNN digit classifier.",
  "endpoints": {
    "GET /health": "Health check.",
    "POST /predict": "Predict the digit for one 8x8 grayscale image. Body: {\\"image\\": [64 numbers, 0-16 or 0-1, row-major 8x8]}",
    "POST /predict/batch": "Predict digits for multiple images. Body: {\\"images\\": [[64 numbers], [64 numbers], ...]}"
  },
  "name": "Digit Classification API"
}

=== GET /health ===
{
  "model_loaded": true,
  "status": "ok"
}

=== POST /predict (valid 8x8 image of a '0'-like pattern, 0-16 scale) ===
{

```

```

    "confidence": 0.9998,
    "predicted_digit": 0,
    "probabilities": {
      "0": 0.9998,
      "1": 0.0,
      "2": 0.0,
      "3": 0.0,
      "4": 0.0,
      "5": 0.0001,
      "6": 0.0,
      "7": 0.0,
      "8": 0.0,
      "9": 0.0
    }
  }
}

=== POST /predict (missing image field -> expect 400) ===
{
  "error": "Missing 'image' field in request body."
}

=== POST /predict (wrong size array -> expect 400) ===
{
  "error": "Expected an 8x8 (64-value) grayscale image, got 3 values."
}

=== POST /predict (invalid JSON body -> expect 400) ===
{
  "error": "Request body must be valid JSON."
}

=== GET /nonexistent (expect 404) ===
{
  "error": "Endpoint not found. See GET / for a list of available endpoints."
}

=== GET /predict (wrong method -> expect 405) ===
{
  "error": "Method not allowed for this endpoint."
}

=== POST /predict/batch (3 real digit images: true labels 3, 7, 8) ===
{
  "count": 3,
  "results": [
    {
      "confidence": 0.9998,
      "predicted_digit": 3,
      "probabilities": {
        "0": 0.0,
        "1": 0.0,
        "2": 0.0,
        "3": 0.9998,
        "4": 0.0,
        "5": 0.0,
        "6": 0.0,
        "7": 0.0,
        "8": 0.0,
        "9": 0.0001
      }
    },
    {
      "confidence": 0.9827,
      "predicted_digit": 7,
      "probabilities": {
        "0": 0.0,
        "1": 0.0007,
        "2": 0.0009,
        "3": 0.0024,
        "4": 0.0022,
        "5": 0.0009,
        "6": 0.0,
        "7": 0.9827,
        "8": 0.0012,
        "9": 0.009
      }
    },
    {
      "confidence": 0.9992,
      "predicted_digit": 8,
      "probabilities": {
        "0": 0.0,

```

```

    "1": 0.0,
    "2": 0.0,
    "3": 0.0006,
    "4": 0.0,
    "5": 0.0,
    "6": 0.0001,
    "7": 0.0,
    "8": 0.9992,
    "9": 0.0001
  }
}
]
}

=== POST /predict/batch (empty list -> expect 400) ===
{
  "error": "'images' must be a non-empty list of 64-value images."
}

```

7. Results and Observations

- All three real test digits (true labels 3, 7, and 8) were classified correctly by the API, with confidence scores between 98.3% and 99.9% - consistent with the ~97% test accuracy measured for this model in Task 1.
- Every error case tested - missing `image` field, wrong-size array, invalid JSON body, an unknown endpoint, and calling an endpoint with the wrong HTTP method - returned a clean, structured JSON error message with the correct HTTP status code (400, 404, or 405) rather than a raw server stack trace.
- The batch endpoint isolates failures per-image: if one image in a batch is invalid, its entry in the response contains an error message while the other, valid images in the same request are still predicted successfully.
- The model is loaded once at application startup rather than on every request, which keeps prediction latency low and avoids repeatedly paying the cost of reading the model file from disk.
- Because the API accepts and returns plain JSON over standard HTTP, it can be called from any client capable of making HTTP requests - not just Python - including web front-ends, mobile apps, or other backend services.

8. Deliverables Produced

Deliverable	File
Flask application source code	app.py
API documentation	README.md
Sample prediction outputs	Section 6.2 of this report; also api_test_log.txt
Sample client script	sample_client.py
Dependency list	requirements.txt
Interactive demonstration notebook	Task4_Flask_API_Demo.ipynb

9. Conclusion

The Task 1 CNN digit-classification model was successfully wrapped in a Flask REST API with single-image and batch prediction endpoints, full JSON request/response handling, and comprehensive error handling for both client-side mistakes (bad input, wrong endpoint, wrong method) and unexpected server-side errors. Live testing against the running server - including genuine images from the original Digits dataset - confirmed the API returns correct, well-formatted predictions and clean, informative error messages in every case tested.